



Interpretable Machine Learning for TabPFN

David Rundel¹ , Julius Kobialka¹ , Constantin von Crailsheim¹ ,
Matthias Feurer^{1,2} , Thomas Nagler^{1,2} , and David Rügamer^{1,2} 

¹ Department of Statistics, LMU Munich, Munich, Germany
{david.rundel,julius.kobialka,constantin.crailsheim,matthias.feurer,
thomas.nagler,david.ruegamer}@stat.uni-muenchen.de

² Munich Center for Machine Learning, Munich, Germany

Abstract. The recently developed Prior-Data Fitted Networks (PFNs) have shown very promising results for applications in low-data regimes. The TabPFN model, a special case of PFNs for tabular data, is able to achieve state-of-the-art performance on a variety of classification tasks while producing posterior predictive distributions in mere seconds by in-context learning without the need for learning parameters or hyperparameter tuning. This makes TabPFN a very attractive option for a wide range of domain applications. However, a major drawback of the method is its lack of interpretability. Therefore, we propose several adaptations of popular interpretability methods that we specifically design for TabPFN. By taking advantage of the unique properties of the model, our adaptations allow for more efficient computations than existing implementations. In particular, we show how in-context learning facilitates the estimation of Shapley values by avoiding approximate retraining and enables the use of Leave-One-Covariate-Out (LOCO) even when working with large-scale Transformers. In addition, we demonstrate how data valuation methods can be used to address scalability challenges of TabPFN. Our proposed methods are implemented in a package `tabpfn_uml` and made available at https://github.com/david-rundel/tabpfn_uml.

Keywords: In-Context Learning · Prior-Data Fitted Networks · Tabular Data · SHAP

1 Introduction

Interpretable machine learning (IML) has become increasingly vital in the field of machine learning (ML), especially when applied to real-world tasks requiring an understanding of the model's predictions. IML serves both as a debugging tool to enhance fairness and ethics, as well as a means to improve social acceptance by fostering trust through interpretability [20].

Recently, Hollmann et al. [14] introduced TabPFN, a prior-data fitted network (PFN) for tabular data that achieves state-of-the-art performance on a diverse range of classification tasks in mere seconds. It approximates probabilistic inference by in-context learning with a Transformer. For small tabular tasks,

TabPFN shows great potential due to its straightforward use without requiring hyperparameter tuning. This renders TabPFN an appealing choice across diverse application domains. TabPFN, like other deep neural networks (DNNs), is however a black box and its missing interpretability makes it less attractive for applications in some research fields, where understanding the predictions can have significant ethical and practical implications [14, 22]. IML implementations have so far focused on classical supervised ML algorithms and are mostly model-agnostic. While there already exist model-specific implementations for certain deep learning (DL) and tree-based models that take advantage of their architectural properties [20], it is not always clear how to apply IML methods to PFNs and TabPFN in particular in a meaningful and efficient manner.

Our Contributions: In this work, we close this gap by modifying existing IML methods to take advantage of the unique properties of TabPFN and thereby enabling efficient computation of IML methods. In particular, these adaptations allow for a more accurate calculation of Shapley values through exact retraining and the use of LOCO, which is usually not a viable option for Transformers [28]. Also, we employ data valuation measures for context optimization, thereby mitigating scalability challenges of TabPFNs. Our methodological contribution is accompanied by a comprehensive, easy-to-use IML toolbox.

2 Supervised Machine Learning and TabPFN

Consider n_{train} independent and identically distributed (*iid*) training observations, gathered in a training data set $\mathcal{D}_P = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{n_{\text{train}}}$ where $\mathbf{x}^{(i)} \in \mathcal{X} \subseteq \mathbb{R}^p$ are p -dimensional feature vectors and $y^{(i)} \in \mathcal{Y} \subseteq \mathbb{R}^g$ are categorical or continuous targets. In the classical supervised ML setting, the goal is to learn the functional relationship between features and targets with a model $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$ from a hypothesis space $\mathcal{H}$ that is fully determined by its parameters $\boldsymbol{\theta}$. For most ML algorithms, given the dataset and a hyperparameter configuration $\boldsymbol{\lambda} \in \Lambda$, a learner or inducer $\mathcal{I}(\mathcal{D}_P, \boldsymbol{\lambda})$ yields such a model by minimizing the empirical risk $\mathcal{R}_{\text{emp}}(\hat{f}(\{\mathbf{x}^{(i)}\}_{i=1}^{n_{\text{train}}}, \{y^{(i)}\}_{i=1}^{n_{\text{train}}}))$. This is accomplished by optimizing the parameters $\boldsymbol{\theta}$ that were either randomly initialized or pre-trained. The resulting model can be used to perform inference $\hat{f}(\{\mathbf{x}^{*(i)}\}_{i=1}^{n_{\text{inf}}})$ on an unseen inference dataset $\{\mathbf{x}^{*(i)}\}_{i=1}^{n_{\text{inf}}}$ of size n_{inf} [3, 17]. In the following, we focus on the binary classification task, i.e., $\mathcal{Y} = \{0, 1\}$, and a probabilistic model $\hat{f}(\cdot) \in [0, 1]$ predicting the probability of the positive class given features and parameters $\boldsymbol{\theta}$. To highlight that $\hat{f}$ is fitted on the set of all available features $P = \{1, \dots, p\}$, we use $\hat{f}_P$ with index P . When the model is fitted on a feature subset $S \subseteq P$, giving rise to a reduced dataset $\mathcal{D}_S = \{\mathbf{x}_S^{(i)}, y^{(i)}\}_{i=1}^{n_{\text{train}}}$, where $\mathbf{x}^{(i)} = \{\mathbf{x}_S^{(i)}, \mathbf{x}_{S^c}^{(i)}\}$ for all $i = 1, \dots, n_{\text{train}}$, we denote the model as $\hat{f}_S$.

For PFNs and TabPFNs [14, 21], the training and inference procedure, however, differs. As opposed to classical supervised ML, where a model is trained

from scratch or fine-tuned on the training subset of a new dataset, PFNs perform inference in a single forward pass. In this approach, a pre-trained neural architecture with fixed parameters is fed with a training set $\mathcal{D}_P$ and inference features $\{\mathbf{x}^{*(i)}\}_{i=1}^{n_{\text{inf}}}$ from a previously unseen task. The model leverages the training set, also known as *context*, to conduct inference on the inference observations (so-called *in-context learning*). We denote the PFN model function as $\hat{f}_{\text{PFN}}(\{\mathbf{x}^{*(i)}\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_P)$, emphasizing that both the inference and training data are supplied during inference, and to highlight that the parameters of PFNs remain unchanged with the training dataset. To achieve this, the underlying model has been pre-trained on artificially generated tasks sampled from a prior distribution. Hence, PFNs are trained in two steps: The first step is the offline training phase, where the model learns to approximate the posterior predictive distribution (PPD) for an inference observation on many synthetic training datasets from diverse tasks (about 10 million datasets in the case of TabPFN). In detail, for every synthetic dataset $\mathcal{D}_{\text{syn}}$, it is trained to replicate the PPD on a query tuple $(Y_{\text{syn}}^{(n_{\text{syn}}+1)}, \mathbf{X}_{\text{syn}}^{(n_{\text{syn}}+1)})$, using $\mathcal{D}_{\text{syn}} \cup (Y_{\text{syn}}^{(n_{\text{syn}}+1)}, \mathbf{X}_{\text{syn}}^{(n_{\text{syn}}+1)})$ as training input. The second step is the inference phase, where a new training set and inference features are fed into the pre-trained network with fixed parameters [14, 22].

TabPFN specifically uses a Transformer model and a prior over tabular classification tasks. The tokens in the Transformer represent the observations, with training observations attending to each other and inference observations only attending to training observations. In particular, there are $\binom{n_{\text{train}}}{2} + n_{\text{train}} \cdot n_{\text{inf}}$ token connections and the computational cost for a forward pass scales as $\mathcal{O}(n_{\text{train}}^2 + n_{\text{train}} \cdot n_{\text{inf}})$. Due to the quadratic complexity in the number of training observations, the recommended maximum size of the training set is 1024 observations [14, 22]. TabPFN eliminates the need for parameter training or hyperparameter tuning during the inference stage, enabling almost instantaneous predictions [22]. Moreover, its unique characteristics include the fact that loss gradients w.r.t. the input data are available in the backward pass, as well as its attention mechanism. Several studies have demonstrated the model’s effectiveness, achieving state-of-the-art performance across various benchmark datasets, specifically for moderately large tabular classification tasks [14, 19].

3 Interpretable Machine Learning for TabPFN

Existing IML implementations have primarily focused on traditional supervised ML algorithms or work for specific DNN architectures not meaningful or applicable to PFNs. In the following, we describe how to adapt existing IML methods to exploit the unique characteristics of TabPFN. This will render the methods more efficient while also preserving their conventional usage paradigms. Our methodological contributions and implementations include local and global methods for assessing feature effects (FEs), feature importance (FI), and data valuation (DV). We have selected the methods according to their popularity, and such that they complement each other w.r.t. their limitations as well as benefit from

TabPFN’s computational efficiency. Where sensible, both continuous and categorical features are supported. In our package, we have not included model-specific Transformer IML methods [2, 5], because the available methods either produce unsuitable (relevance) scores or cannot be applied to TabPFN due to their architecture. Table 1 summarizes the methods and modifications made in this paper and implemented in the package `tabpfn_iml`.

Table 1. Overview of methods adapted and modified for TabPFN. In addition to FE, FI, and DV, we also include methods for clinical value assessment (CVA), uncertainty quantification (UQ), and local explanations (LE).

Method	Type	Scope	Modifications
ICE (Sect. 3.1)	FE	local	Improved efficiency
PD (Sect. 3.1)	FE	global	Improved efficiency
ALE (Sect. 3.1)	FE	global	Improved efficiency
Kernel SHAP (Sect. 3.3)	FE	local & global	Exact retraining
SA (Sect. 3.5)	FE	local & global	–
LOCO (Sect. 3.2)	FI	global	Rendered tractable
SAGE (Sect. 3.6)	FI	global	Exact retraining
LOO (Sect. 3.4)	DV	global	Rendered tractable
Data Shapley (Sect. 3.4)	DV	global	Exact retraining
SA (Sect. 3.5)	DV	local & global	–
DCA (Sect. 3.6)	CVA	global	–
CP (Sect. 3.6)	UQ	local	–
MOC (Sect. 3.6)	LE	local	–

3.1 ICE, PD and ALE

Individual conditional expectation (ICE) curves [13, 20], partial dependence (PD) plots [10, 20] and accumulated local effect (ALE) plots [1, 20] show how the predictions of a model change along a feature space of interest by systematically varying the respective input values. This necessitates prediction on artificial observations, where the feature of interest is varied according to a specified grid generated by discretizing the feature space into G grid points. PD approximates the expected target response, while ALE addresses the extrapolation issue of ICE and PD [20].

Each TabPFN forward pass involves $n_{\text{train}}((n_{\text{train}} - 1)/2 + n_{\text{inf}})$ token connections (cf. Sect. 2) and thus the computational cost per inference sample scales as $\mathcal{O}(n_{\text{train}}^2/n_{\text{inf}})$. Consequently, TabPFN yields runtime improvements when processing as many inference samples as possible in a single forward pass. For the PD plot, widely used implementations such as the scikit-learn `partial_dependence`

function, execute a prediction step for every point of the grid. Instead, we propose a more efficient way to compute the PD plot in just a single forward pass by constructing a large inference array that contains all artificial observations. Compared to the scikit-learn implementation which has computational complexity $\mathcal{O}((n_{\text{train}}^2 + n_{\text{train}} \cdot n_{\text{inf}}) \cdot G)$, our implementation then scales as $\mathcal{O}(n_{\text{train}}^2 + n_{\text{train}} \cdot n_{\text{inf}} \cdot G)$. Thus, our PD plot implementation implies substantially lower runtimes. The modifications and implementations of ICE and ALE follow the same logic.

3.2 LOCO

Leave-One-Covariate-Out (LOCO) estimates the effect of features on the predictive performance of a learner by retraining it on a subset of features without the feature of interest [16, 20]. The starting point is a model $\hat{f}_P$ trained with all features. Then, for each feature of interest j , the feature is removed from the dataset ($\mathcal{D}_{P \setminus \{j\}}$), the model is retrained without it ($\hat{f}_{P \setminus \{j\}}$) and the resulting change in empirical risk is tracked. LOCO approximately indicates whether a feature contains unique information that no other feature contains [16, 20].

For many DL models, particularly extensive Transformer architectures, calculating LOCO scores is not feasible due to the high computational cost of repeatedly retraining models on different feature subsets. However, we can easily compute LOCO for TabPFN. Due to in-context learning, TabPFN enables inference on an updated training set through a single forward pass. In particular, we propose to calculate LOCO for the j -th feature as

$$\text{LOCO}_j = \mathcal{R}_{\text{emp}}(\hat{f}_{\text{PFN}}(\{\mathbf{x}_{P \setminus \{j\}}^{*(i)}\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_{P \setminus \{j\}}), \{y^{*(i)}\}_{i=1}^{n_{\text{inf}}}) \\ - \mathcal{R}_{\text{emp}}(\hat{f}_{\text{PFN}}(\{\mathbf{x}_P^{*(i)}\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_P), \{y^{*(i)}\}_{i=1}^{n_{\text{inf}}}).$$

3.3 Kernel SHAP

Shapley values [24] have their origin in game theory and aim to fairly attribute payouts. In ML, they estimate the local FE for an observation $\mathbf{x}^{*(i)}$ in a model $\hat{f}_P$ [20]. In detail, the Shapley value for the j -th feature considers the (weighted) average marginal contribution of the feature to the model prediction across all feature subsets. While Shapley values provide a theoretically sound approach to attribution considering feature interactions, their calculation can be costly since the learner needs to be retrained for each feature subset S . As retraining ML models is in general resource-intensive and given that there are 2^p distinct feature subsets, this becomes impractical for most learners and datasets.

To address these challenges, Kernel SHapley Additive exPlanations (Kernel SHAP) [17] seeks to provide tractable approximations to Shapley values for a more efficient estimation. It employs an additive feature attribution measure by fitting a weighted linear model as local surrogate model that regresses model predictions on a binary representation of feature subsets, where only the features that appear in the subset are used for prediction [17, 20]. To make the computations tractable, it incorporates the following additional approximations:

- M feature subsets are randomly sampled, as opposed to exhaustive consideration of all 2^p subsets. M serves as a hyperparameter that balances computational cost and approximation quality [17, 26].
- To avoid retraining the learner on every sampled feature subset S , retraining is approximated by using the model trained on all features $\hat{f}_P$. This approach involves marginalizing over the features S^C that are absent in a subset while assuming that all features are independent and approximating an integral by Monte Carlo.¹ Specifically, the approximation can be expressed as

$$\hat{f}_S(\cdot) \approx L^{-1} \sum_{l=1}^L \hat{f}_P((\cdot, \mathbf{x}_{S^C}^{\kappa(l)})). \quad (1)$$

The indices $\kappa(l) \in \{1, \dots, n_{\text{train}}\}$ for $l \in \{1, \dots, L\}$ are randomly sampled and referred to as *imputation samples*, used to impute S^C [17, 26].

We can exploit the unique characteristics of TabPFN to enhance Kernel SHAP by obtaining exact solutions, rather than approximations, for certain terms in the Shapley value estimation. First, consider a naïve implementation of Kernel SHAP coupled with TabPFN. Using Eq. 1, the approximation for retraining TabPFN on a feature subset S is given by

$$\hat{f}_{\text{PFN}}(\{\mathbf{x}_S^{*(i)}\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_S) \approx L^{-1} \sum_{l=1}^L \hat{f}_{\text{PFN}}(\{(\mathbf{x}_S^{*(i)}, \mathbf{x}_{S^C}^{\kappa(l)})\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_P).$$

An *approximate retraining* implementation could process all n_{inf} inference samples and M feature subsets in parallel using L imputation samples, scaling as

$$\mathcal{O}(n_{\text{train}}^2 + n_{\text{train}} \cdot n_{\text{inf}} \cdot M \cdot L). \quad (2)$$

However, since training and prediction are done jointly in TabPFN’s forward pass, $\hat{f}_{\text{PFN}}(\{\mathbf{x}_S^{*(i)}\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_S)$ does not need to be approximated. Instead, we iterate over M feature subsets, for each restricting the features of both the training and inference set accordingly, and evaluate the model for all n_{inf} inference samples. We refer to this approach as *exact retraining*, scaling as

$$\mathcal{O}((n_{\text{train}}^2 + n_{\text{train}} \cdot n_{\text{inf}}) \cdot M). \quad (3)$$

Assuming $n_{\text{train}} \approx n_{\text{inf}}$ and $L \geq 2$, our method has thus lower computational cost. In Eq. 2, we assume that all calculations can be executed in a single forward pass. As this is mostly not possible due to memory constraints on commonly used hardware, we also provide an *approximate retraining* implementation, where we iterate over M and L , predicting sequentially.

3.4 Data Valuation for Context Optimization

One of the primary limitations of TabPFN is its scalability in n_{train} . As a consequence, its context is restricted to approximately 1000 observations. This limits the widespread adoption of TabPFN, especially for larger datasets [9].

¹ In this work, we chose not to consider the optional model linearity assumption as it is overly simplistic.

A straightforward way to use TabPFN with larger datasets involves random sketching. This entails selecting a random subset of size $n_{\text{sub}} \leq 1024$ from the complete training set as context. In contrast, context optimization involves using more advanced strategies to determine appropriate contexts when dealing with large datasets. Recently, researchers proposed prompt-tuning strategies to learn an optimal context via gradient descent [9, 18]. In our work, we pursue a different approach to context optimization by selecting a representative training subset based on IML methods. Various data valuation techniques, e.g., Data Shapley, which quantify the contribution of training observations to the predictive performance, have been suggested in the literature. Empirical evidence further suggests that incorporating training samples based on data values can enhance performance of ML models [12].

In contrast to Shapley values for FEs, Data Shapley considers the marginal contributions of training observations – as opposed to features – to the model performance across all training subsets. As retraining is impractical for complex learners and datasets, several approximations such as Gradient Shapley [12] have been proposed. Gradient Shapley employs simplistic approximations to estimate the marginal contributions, such as utilizing stochastic gradient descent with a batch size of one. Some advancements exist, but most of these involve additional approximations and yet require training multiple models, limiting their applicability [15, 23, 29]. We can, however, implement Data Shapley as a modification of Kernel SHAP with exact retraining by making use of in-context learning. This eliminates the reliance on such simplistic approximations. Consider a training dataset of size $n_{\text{train}} > n_{\text{sub}}$ (typically $n_{\text{sub}} = 1024$) where our objective is to identify an optimal subset of size n_{sub} . To achieve this, we evaluate M randomly sampled training subsets, ranging in size from 256 (for sufficient context) to n_{sub} , on a validation set of size n_{val} . The resulting empirical risk values are then attributed to the training set observations by fitting a weighted linear model as a local surrogate. Subsequently, we select the n_{sub} training observations with lowest coefficients in the linear model as the context, expecting that these will contribute the least to the increase in empirical risk. Kernel SHAP forgoes the need to examine all $2^{n_{\text{train}}}$ training subsets. In order to ensure estimation of the surrogate model, we, however, require $M \geq n_{\text{train}}$, leading to substantial computational expenses.

Furthermore, TabPFNs architecture facilitates the incorporation of additional DV techniques, including Leave-one-out (LOO) testing [6]. Similar to LOCO, LOO removes a single training observation from the training procedure and computes the change in predictive performance [23, 29].

3.5 Sensitivity Analysis

Sensitivity Analysis (SA) is a model-specific IML method that leverages gradients computed in a backward pass to extract relevance scores for features [11]. Computing SA for FE is straightforward for TabPFN and also included in our implementation. Leveraging the differentiability of TabPFN w.r.t. training

observations, we follow a similar approach to adapt SA for data values. For the data value of the j -th training observation, we compute the gradient

$$R_j = \left\| \nabla_{(\mathbf{x}^{(j)}, y^{(j)})} \mathcal{R}_{\text{emp}}(\hat{f}_{\text{PFN}}(\{\mathbf{x}^{*(i)}\}_{i=1}^{n_{\text{inf}}}, \mathcal{D}_P), \{y^{*(i)}\}_{i=1}^{n_{\text{inf}}}) \right\|_2$$

of the empirical risk w.r.t. that observation and aggregate it to a scalar using the L2-norm. This allows identifying training observations for which the empirical risk is highly sensitive.

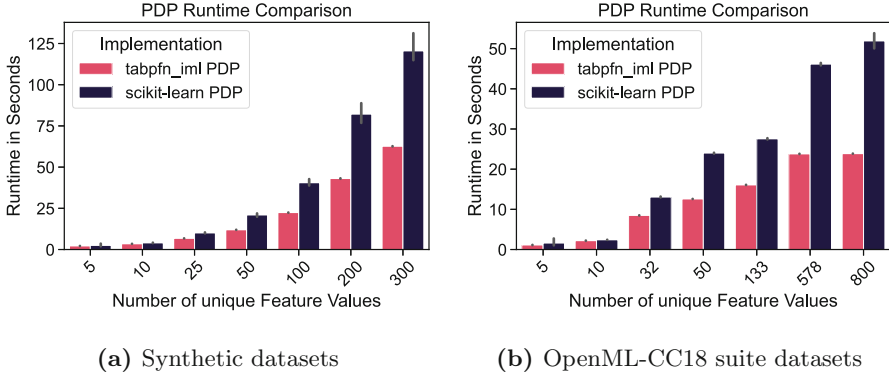


Fig. 1. Runtime comparison of the implementations of `tabpfn_uml` PD vs. `scikit-learn partial_dependence()`. (Color figure online)

3.6 Further Methods

In order to complement our toolbox to provide a comprehensive IML tool, we include Shapley Additive Global Importances (SAGE) [7] as an additional FI method. Our implementation involves utilizing Kernel SHAP with the empirical risk as the function to be explained and also benefits from exact retraining. We also offer wrappers for decision curve analysis (DCA), a CVA method as implemented in [25], and conformal prediction (CP), an UQ method as implemented in [27]. Last but not least, we implement multi-objective counterfactual explanations (MOC) from the realm of LE using evolutionary multi-objective algorithms (EMOAs) [8]. For more information, we refer to the [tabpfn_uml repository](#).

4 Numerical Experiments²

In the following, we provide numerical evidence of the improvement in computational efficiency and approximation error obtained with our modifications and implementation. Furthermore, we show how DV methods can be employed to improve classification performance by context optimization.

² All experiments were run on a Nvidia GTX A6000 GPU. Furthermore, our experiment code in the repository enables the replication of all results.

4.1 ICE and PD

To show the improved efficiency of our ICE and PD implementation, we conduct a runtime comparison across 25 synthetic datasets with 10 features and 1000 observations in Fig. 1a and across 7 binary classification datasets with increasing size from the OpenML-CC18 suite in Fig. 1b. For the synthetic data, we define a complex non-linear relationship between 10 features and a binary target, add a random noise term, and discretize the feature of interest to the number of intervals indicated on the x -axis, as detailed in our GitHub repository. We compare our implementation to scikit-learn’s `partial_dependence()` across five different seeds per feature level, setting the `grid_size` equal to the number of unique feature values. In Fig. 1b, we set the `grid_size` to the default value of 100 to depict runtimes for different real-world datasets. For both Figures, n_{train} comprises 80% of all observations, and n_{inf} the remaining 20%. Our modifications to ICE and PD result in much better runtime, with our implementation being substantially faster as the grid size or data set size increase.

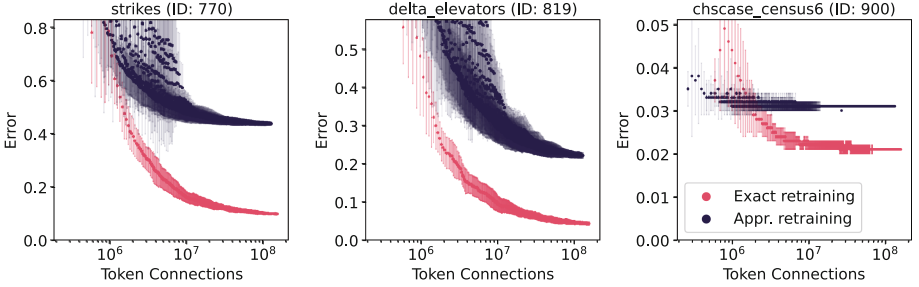


Fig. 2. Average errors (points) and corresponding standard deviations (error bars) across all runs plotted against the token connections for various configurations and three datasets (different plots). Exact retraining configurations are red, while approximate retraining configurations are black. (Color figure online)

4.2 Kernel SHAP

Next, we empirically compare our proposed exact retraining method to the alternative approximate method. The comparison of the two Kernel SHAP approaches is done by defining a (stochastic) error metric, which for each calculates the sum of the absolute deviations from the exactly computed Shapley values $\phi_j(\hat{f}_{\text{PFN}}, \mathbf{x}^{*(i)})$ across features j , averaged over inference observations i :

$$\frac{1}{n_{\text{inf}}} \sum_{i=1}^{n_{\text{inf}}} \sum_{j=1}^P |\phi_j(\hat{f}_{\text{PFN}}, \mathbf{x}^{*(i)}) - \hat{\phi}_{j,(M,L)}(\hat{f}_{\text{PFN}}, \mathbf{x}^{*(i)})|.$$

$\hat{\phi}_{j,(M,L)}(\hat{f}_{\text{PFN}}, \mathbf{x}^{*(i)})$ indicates the estimated Shapley value for the j -th feature and i -th inference observation. We further specify $M \in \{1, 2, \dots\}$ and $L \in$

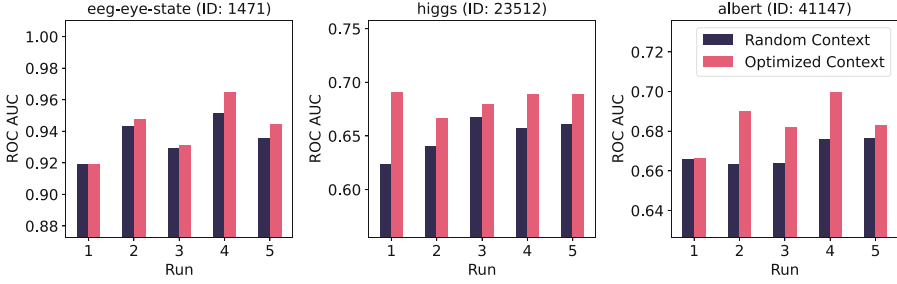


Fig. 3. Comparison of the ROC AUC of randomly selected TabPFN contexts (black) and contexts optimized through Data Shapley coupled with Kernel SHAP (red) across five runs and three datasets. (Color figure online)

$\{-1, 1, 2, \dots\}$ as the number of feature subsets and imputation samples, where we denote exact retraining as $L = -1$. Since the exact computation of Shapley values scales exponentially in the number of features, we select three binary classification tasks from the OpenML-CC18 suite [4] with six numerical features, aligning with the optimal application scenario for TabPFN [14]. For each dataset, we randomly sample a training set of size $n_{\text{train}} = 256$ and an inference set of size $n_{\text{inf}} = 128$ and evaluate a fine-grained grid across all values of $M \in \{6, \dots, 160\}$ and $L \in \{1, \dots, 25\}$ for approximate retraining, and across values of $M \in \{6, \dots, 1600\}$ for exact retraining. We repeat all experiments 25 times for each dataset, with feature subsets and imputation samples randomly sampled in each iteration.

In Fig. 2, we present the mean and one standard deviation of the error across all runs, contrasted with the corresponding number of token connections for each (M, L) configuration according to Eqs. 2 and 3. The plot shows that exact retraining almost exclusively achieves lower approximation error for a fixed computational budget. In contrast, achieving comparable error values with approximate retraining often demands a larger number of feature subsets and multiple imputation samples. Moreover, exact retraining converges at a lower error and the standard deviations indicate that its Shapley estimates have lower variance. On top, it is important to stress that the number of token connections for approximate retraining is a lower bound and approximate retraining converges even slower in practice. Notably, for the chscase.census6 dataset, there are configurations with a small number of token connections where approximate retraining outperforms the exact equivalent, which might be attributed to the cost-effectiveness of approximate retraining for low values of L .

4.3 Data Valuation for Context Optimization

We conduct an empirical evaluation of our Data Shapley implementation for context optimization by comparing it to random sketching. Our benchmark consists of three datasets from the OpenML-CC18 suite [4], chosen in accordance with

[9], suggesting that TabPFN’s performance on these datasets can be enhanced through context optimization. For each dataset, we create a training set of size $n_{\text{train}} = 3072$ and aim to identify an optimal context of size $n_{\text{sub}} = 512$ thereof. We choose $M = 3 \cdot n_{\text{train}}$, $n_{\text{val}} = 512$ and ultimately assess the optimized context’s performance on a previously unseen test set of size $n_{\text{test}} = 1024$, comparing it to the performance of randomly selected contexts of the same size. This procedure is repeated 5 times with differently sampled subsets.

In Fig. 3, we compare the ROC AUC on the test set of both random and optimized contexts across all runs and three datasets. The plot indicates that our method consistently outperforms random sketching. On the eeg-eye-state and albert datasets, we observe an average increase in ROC AUC of 0.57% and 1.52%, respectively. An even more substantial improvement is obtained for the higgs dataset, yielding a difference of 3.3% in ROC AUC. The results suggest that our implementation of Data Shapley produces sensible results, indicating its potentially meaningful application in domains beyond context optimization. However, note that this procedure requires nearly 10,000 TabPFN forward passes and is – in contrast to our other proposed methods – notably more expensive.

5 Discussion and Conclusion

By exploiting the architecture underlying TabPFN and in-context learning, we propose tailored modifications and adaptations of IML methods for TabPFN. Specifically, we provide efficient implementations of ICE, PD, and ALE. Furthermore, using similar insights, we make the computation of LOCO and LOO possible despite TabPFN’s Transformer nature. Moreover, we have shown that our exact retraining approach to Kernel SHAP dominates current alternatives with respect to both approximation error and runtime. Finally, our implementation of Data Shapley yields meaningful results that can be used for context optimization. Together with further methods in IML, we provide a comprehensive and easy-to-use IML package that allows to shed light on TabPFN’s predictions. The suggested modifications to IML methods also translate to the broader realm of in-context learning, especially for tabular data, and exploring their potential is a promising direction for future research.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article

References

1. Apley, D.W., Zhu, J.: Visualizing the effects of predictor variables in black box supervised learning models. *J. R. Stat. Soc.* **82**(4), 1059–1086 (2020)
2. Bastings, J., Filippova, K.: The elephant in the interpretability room: why use attention as explanation when we have saliency methods? In: *EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP* (2020)
3. Bischl, B., et al.: Hyperparameter optimization: foundations, algorithms, best practices, and open challenges. *Data Min. Knowl. Discov.* **13**(2), e1484 (2023)

4. Bischl, B., et al.: Openml benchmarking suites. In: *NeurIPS Datasets and Benchmarks Track*, vol. 1 (2021)
5. Chefer, H., Gur, S., Wolf, L.: Transformer interpretability beyond attention visualization. In: *CVPR*, pp. 782–791 (2021)
6. Cook, R.D.: Detection of influential observation in linear regression. *Technometrics* **19**(1), 15–18 (1977)
7. Covert, I., Lundberg, S.M., Lee, S.I.: Understanding global feature contributions with additive importance measures. In: *NeurIPS*, vol. 33, pp. 17212–17223 (2020)
8. Dandl, S., Molnar, C., Binder, M., Bischl, B.: Multi-objective counterfactual explanations. In: Bäck, T., et al. (eds.) *PPSN 2020. LNCS*, vol. 12269, pp. 448–469. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58112-1_31
9. Feuer, B., et al.: Tunetables: context optimization for scalable prior-data fitted networks. [arXiv:2402.11137](https://arxiv.org/abs/2402.11137) (2024)
10. Friedman, J.H.: Greedy function approximation: a gradient boosting machine. *Ann. Stat.* 1189–1232 (2001)
11. Gevrey, M., Dimopoulos, I., Lek, S.: Review and comparison of methods to study the contribution of variables in artificial neural network models. *Ecol. Model.* **160**(3), 249–264 (2003)
12. Ghorbani, A., Zou, J.: Data shapley: equitable valuation of data for machine learning. In: *ICML*, vol. 97, pp. 2242–2251. PMLR (2019)
13. Goldstein, A., Kapelner, A., Bleich, J., Pitkin, E.: Peeking inside the black box: visualizing statistical learning with plots of individual conditional expectation. *J. Comput. Graph. Stat.* **24**(1), 44–65 (2015)
14. Hollmann, N., Müller, S., Eggensperger, K., Hutter, F.: TabPFN: a transformer that solves small tabular classification problems in a second. In: *ICLR* (2023)
15. Kwon, Y., Zou, J.: Data-OOB: out-of-bag estimate as a simple and efficient data value. In: *ICML*, vol. 202, pp. 18135–18152. PMLR (2023)
16. Lei, J., G’Sell, M., Rinaldo, A., Tibshirani, R.J., Wasserman, L.: Distribution-free predictive inference for regression. *J. Am. Stat. Assoc.* **113**(523), 1094–1111 (2018)
17. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: *NeurIPS*, vol. 30 (2017)
18. Ma, J., Thomas, V., Yu, G., Caterini, A.: In-context data distillation with TabPFN. *arXiv preprint [arXiv:2402.06971](https://arxiv.org/abs/2402.06971)* (2024)
19. McElfresh, D., et al.: When do neural nets outperform boosted trees on tabular data? In: *NeurIPS Datasets and Benchmarks Track*, vol. 36 (2024)
20. Molnar, C.: *Interpretable machine learning*, 2 edn. (2020)
21. Müller, S., Hollmann, N., Arango, S.P., Grabocka, J., Hutter, F.: Transformers can do bayesian inference. In: *ICLR* (2022)
22. Nagler, T.: Statistical foundations of prior-data fitted networks. In: *ICML*, vol. 202, pp. 25660–25676. PMLR (2023)
23. Rozenberczki, B., et al.: The shapley value in machine learning. In: *IJCAI*, pp. 5572–5579 (2022)
24. Shapley, L.S.: A Value for N-Person Games. RAND Corporation (1952)
25. Sjöberg, D.D.: dcurves: decision curve analysis for model evaluation (2022)
26. Štrumbelj, E., Kononenko, I.: Explaining prediction models and individual predictions with feature contributions. *Knowl. Inf. Syst.* **41**, 647–665 (2014)
27. Taquet, V., Blot, V., Morzadec, T., Lacombe, L., Brunel, N.: MAPIE: an open-source library for distribution-free uncertainty quantification. [arXiv:2207.12274](https://arxiv.org/abs/2207.12274) (2022)
28. Vaswani, A., et al.: Attention is all you need. In: *NeurIPS*, vol. 30 (2017)
29. Wang, J.T., Jia, R.: Data banzhaf: a robust data valuation framework for machine learning. In: *AISTATS*, vol. 206, pp. 6388–6421. PMLR (2023)